



KV Cache

在自注意力中，当模型要生成下一个**token**时，会关注所有已经生成的词。



多头注意力中的Q、K、V如何随着生成长度增加而增加

💡 每生成一个新词，都会新增一个 token，它会产生新的 Q、K、V；同时，历史的 K 和 V 也会一直保存，供后续所有新词使用。

例子：假设使用 4 个注意力头（Head），隐藏维度为 d ，生成序列：<BOS> - 今 - 天 - 天 - 气 - 真 - 不 - 错 - ， - 我 -



KV Cache:把已经计算过的历史词的K(键)和V(值)保存起来，后续生成新词时直接复用，不需要重新计算，从而大幅提升效率，节省计算和显存。

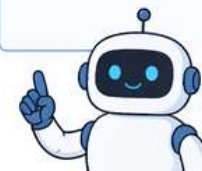
例子：生成句子

<BOS> 今 天 天 气 真 不 错 ， 我 们



关键说明

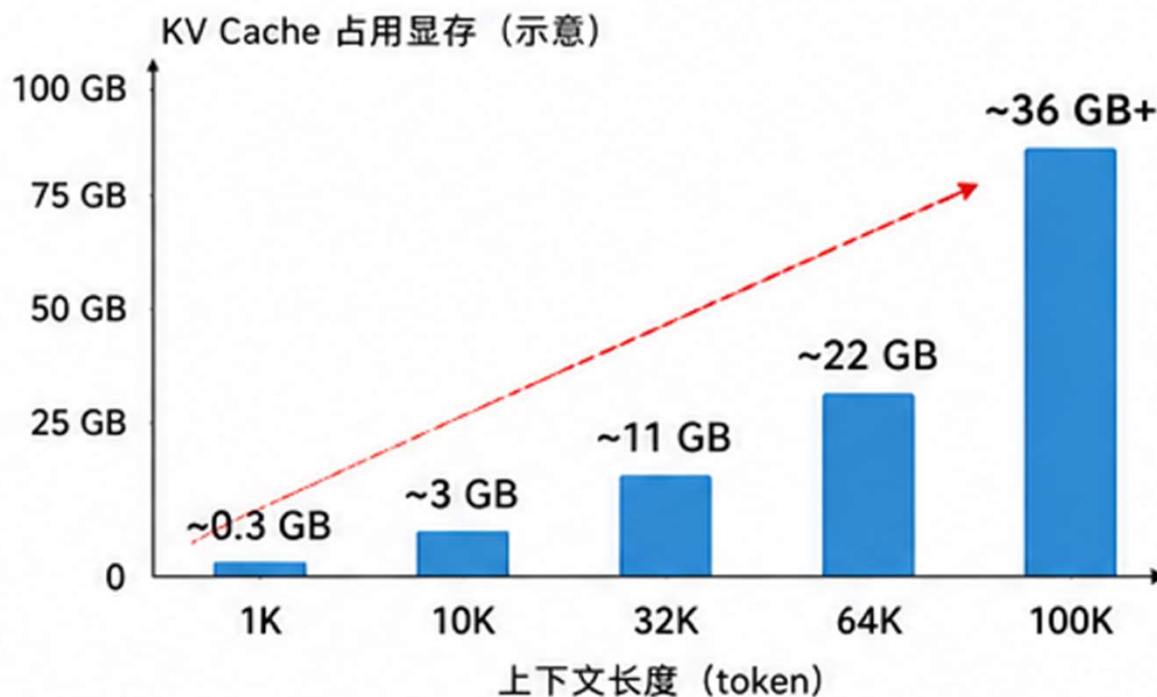
- 第 t 步生成时，KV Cache 中保存的是前 t 个 token 的 K、V (包括 <BOS>)，也就是位置 $0 \sim t-1$ 。
- 当前步骤只计算当前位置的 Q，与所有历史位置的 K、V 计算注意力。
- 当前 token 的 K、V 只有在该 token 生成完成之后才写入 Cache，供后续步骤复用。
- 这样，计算量从 $O(t^2)$ 降到 $O(t)$ 。

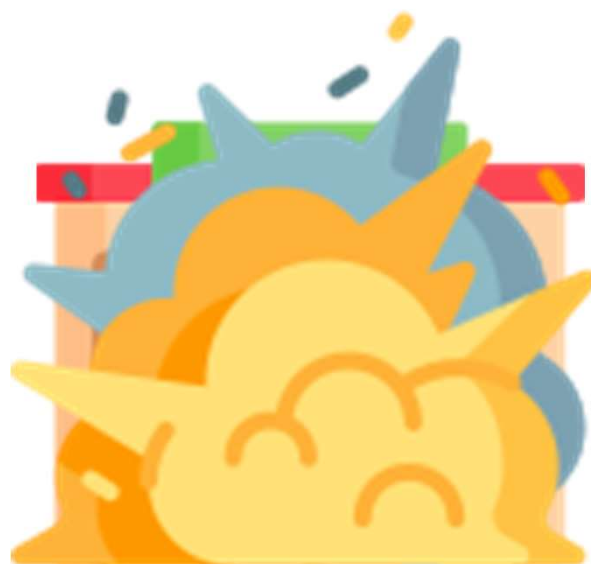


KV Cache会随着上下文长度线性增长

每增加 1 个 token,
都要新增一份 K 和 V,
显存占用**持续**增加。

线性增加



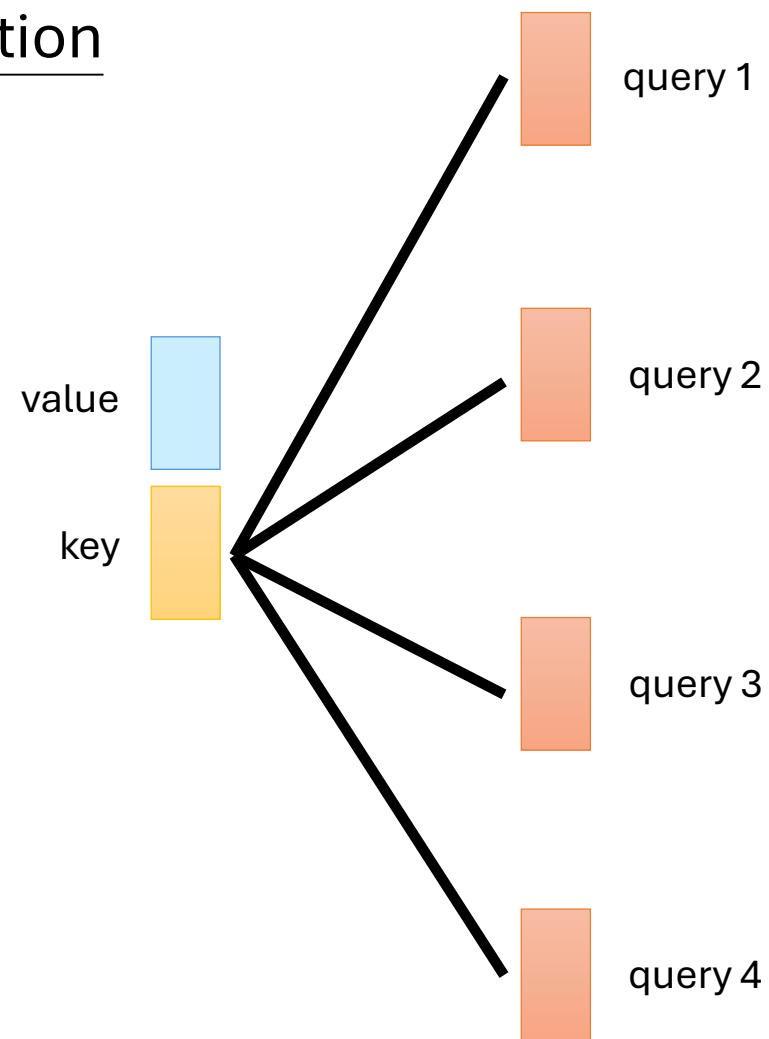
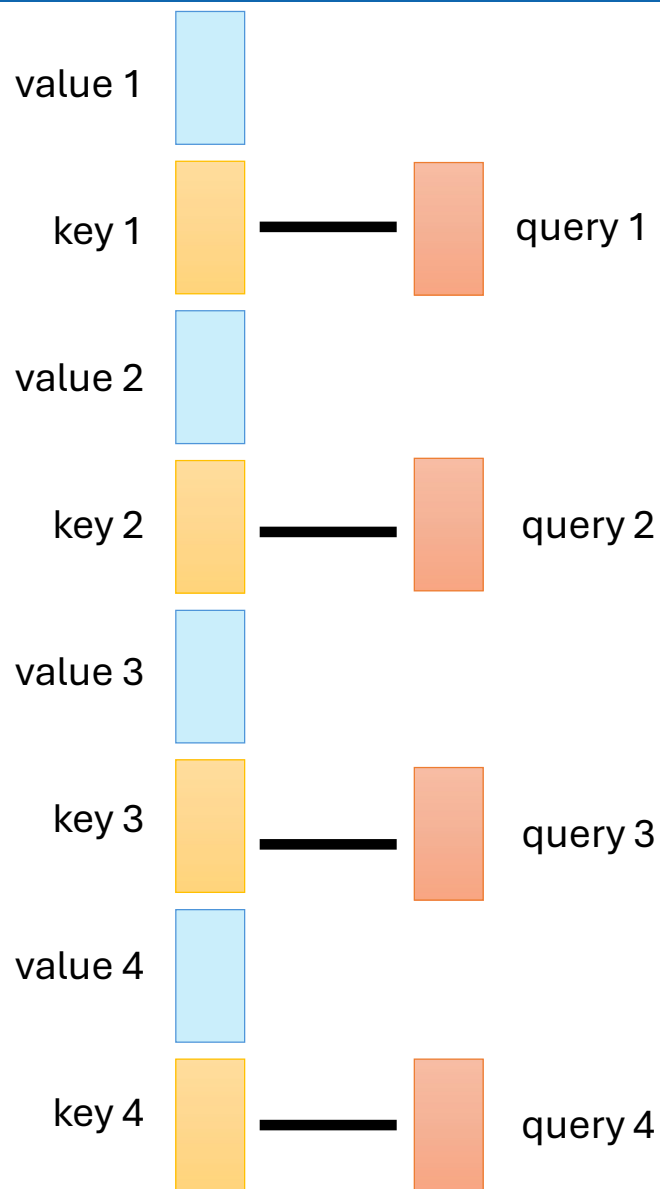


HBM



SRAM

Multi-query att ention

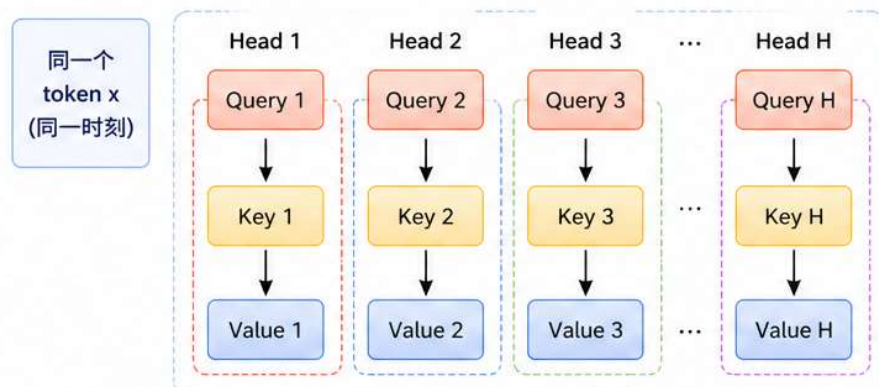


Multi-query attention: 多个Query Heads, 共享一套K/V Heads

💡 注意：这里的多个 Query Head 指的是同一个 token（同一时刻）的不同注意力头，而不是不同的 token。

1. Multi-Head Attention (MHA)

每个 Head 拥有**独立**的 K、V



结果：H 个 Query Head 需要 **H 套** 独立的 K/V

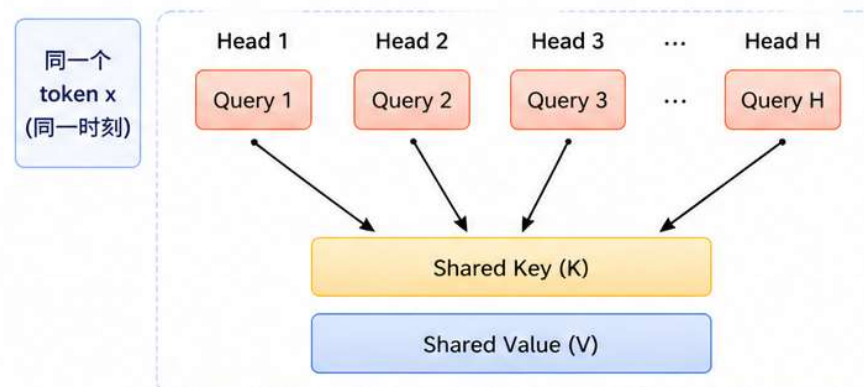
推理阶段 KV Cache 占用 (MHA)



占用显存 $\propto$ Head 数 (H) $\times$ 序列长度 (T) $\times$ 维度 (D) = $H \times T \times D$

2. Multi-Query Attention (MQA)

所有 Query Head **共享** 同一套 K、V



结果：H 个 Query Head 只需要 **1 套** K/V

推理阶段 KV Cache 占用 (MQA)

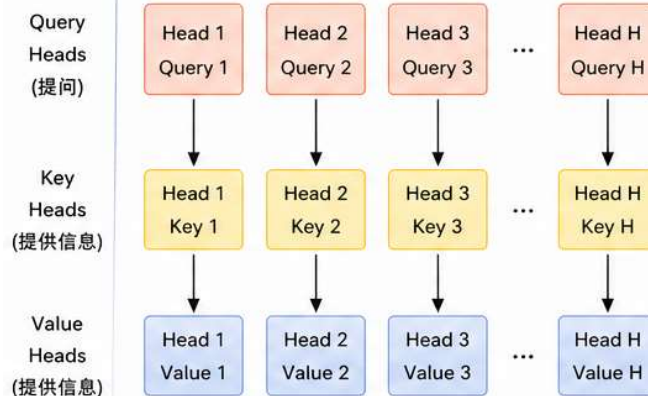


占用显存 $\propto$ 1 $\times$ 序列长度 (T) $\times$ 维度 (D) = $1 \times T \times D$ (比 MHA **减少 H 倍!**)

Group-query attention

1. Multi-Head Attention (MHA) 传统多头注意力

每个 Query Head 拥有独立的 Key/Value Head



特点: H 个 Query Head 需要 H 套独立的 K/V

KV Cache 占用 (推理阶段)

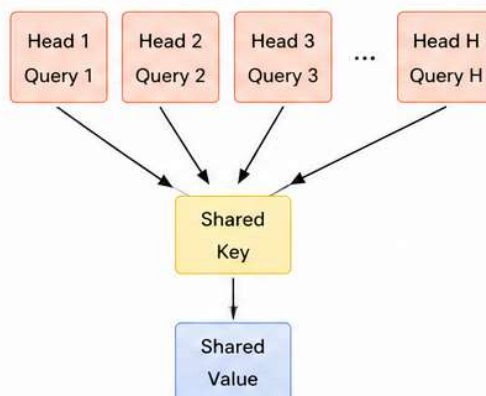


占用显存 $\propto H \times T \times D$

H: Query Head 数; T: 序列长度; D: Head 维度

2. Multi-Query Attention (MQA) 多查询注意力

所有 Query Head 共享同一套 Key/Value Head



特点: H 个 Query Head 只需要 1 套 K/V

KV Cache 占用 (推理阶段)

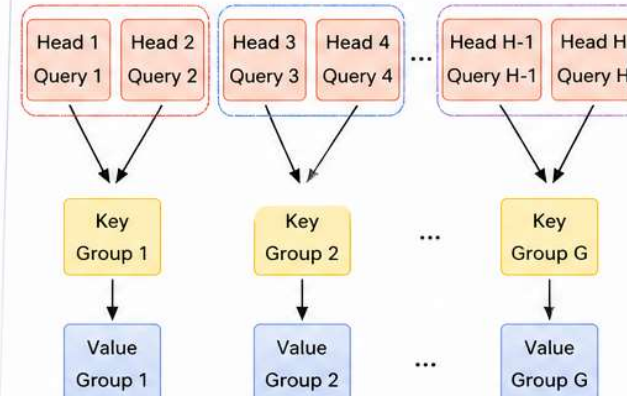


占用显存 $\propto 1 \times T \times D$

比 MHA 减少 H 倍!

3. Group-Query Attention (GQA) 分组查询注意力

将 Query Head 分成 G 组, 每组共享一套 K/V Head



特点: H 个 Query Head 只需要 G 套 K/V ($1 < G < H$)

KV Cache 占用 (推理阶段)



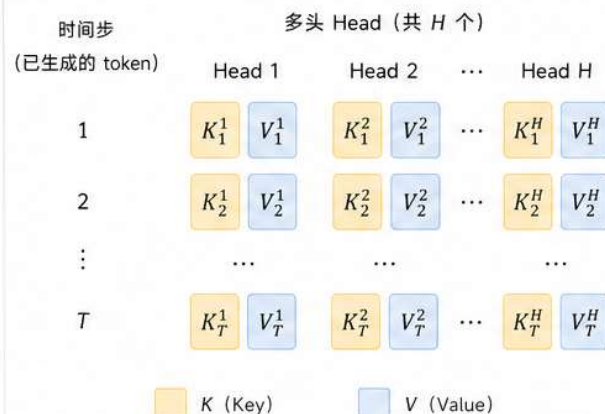
占用显存 $\propto G \times T \times D$

比 MHA 减少 H/G 倍!

Multi-head Latent Attention

不再为每个head保存完整的K和V，而是把每个token的隐藏表示h压缩成一个更小的latent向量c存入缓存。推理时再从c恢复出K和V来使用，既节省显存，又保留必要的信息。

1. 传统做法：为每个 head 保存完整的 K 和 V



传统 KV Cache 的存储量

$$O(H \times T \times d_{kv})$$

H: head 数; T: 序列长度; d_{kv} : 每个 K/V 向量的维度

2. MLA 做法：从隐藏表示直接压缩为 latent 向量 c



MLA Cache 的存储量

$$O(T \times d_c)$$

其中 $d_c \ll H \times d_{kv}$, 显存大幅降低!

3. 缓存结果：只存更小的 latent 向量 c

Latent Cache (实际存储的内容)

c_1
 c_2
 c_3
 $\vdots$
 c_T

只与 T 和 d_c 有关,
与 head 数 H 无关!

✓ MLA 的优势

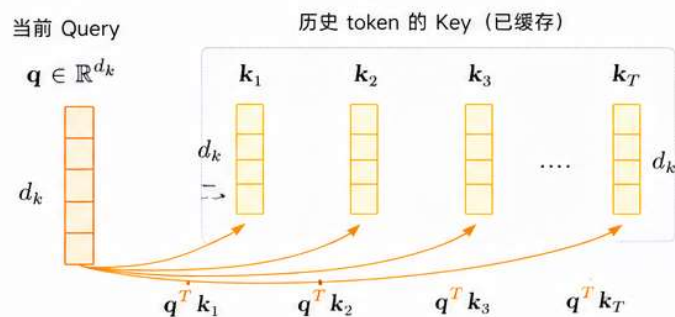
- ✓ 显存占用大幅降低 (不随 head 数增长)
- ✓ 支持更长上下文
- ✓ 提升并发能力
- ✓ 推理更高效

存储量对比 (相对减少倍数)

$$\frac{O(T \times d_c)}{O(H \times T \times d_{kv})} = \frac{d_c}{H \times d_{kv}} \ll 1$$

Multi-head Latent Attention

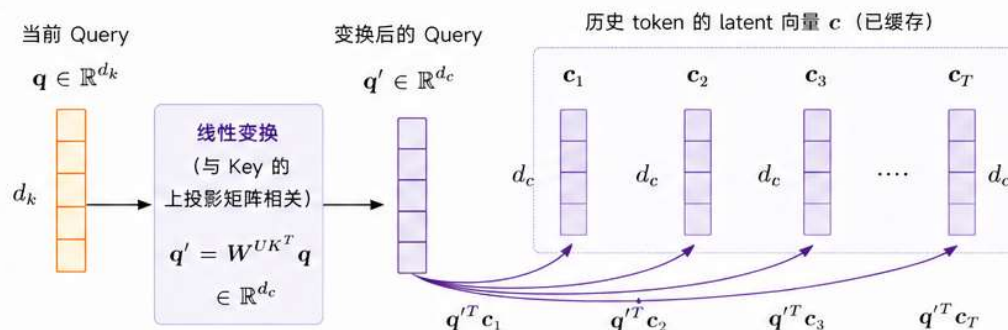
1. 传统注意力：用 Key 计算得分



注意力得分：

$$score_i = q^T k_i \in \mathbb{R}, \quad i = 1, \dots, T$$

2. MLA: 用 latent 向量 c 计算得分



注意力得分：

$$score'_i = q'^T c_i \in \mathbb{R}, \quad i = 1, \dots, T$$

3. 为什么两种打分等价？（数学推导）

MLA 中，latent 向量 c_i 可以线性恢复 Key:

$$k_i = W^{UK} c_i, \quad W^{UK} \in \mathbb{R}^{d_k \times d_c} \quad (\text{公式 10})$$

将其代入传统打分公式：

$$\begin{aligned} score_i &= q^T k_i = q^T (W^{UK} c_i) \\ &= (W^{UK^T} q)^T c_i \\ &= q'^T c_i \end{aligned}$$

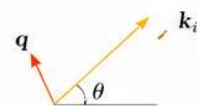
$$\text{令 } q' = W^{UK^T} q$$

$$\text{因此: } score_i = q^T k_i = q'^T c_i = score'_i$$

4. 直观理解

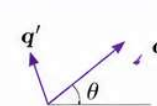
几何视角（内积不变性）

传统：在 d_k 维空间中计算



$$\text{打分} = |q| |k_i| \cos \theta$$

MLA：在 d_c 维空间中计算



$$\text{打分} = |q'| |c_i| \cos \theta$$

由于 k_i 与 c_i 线性相关，角度信息被保留（在子空间中），因此打分等价。

总结一句话

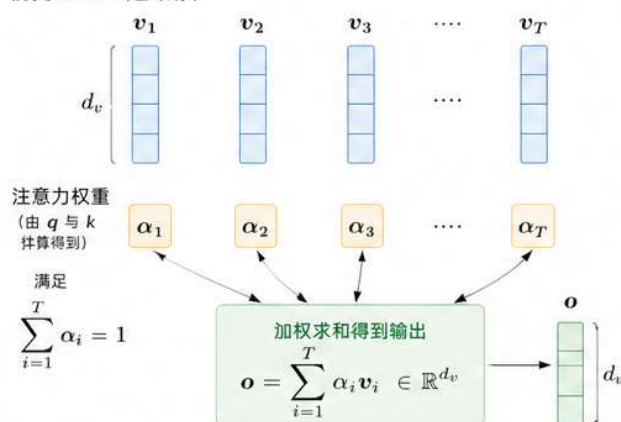
只要 $k_i = W^{UK} c_i$
就可以把
“与 Key 打分”
变成
“与 c 打分”
完全等价！



Multi-head Latent Attention

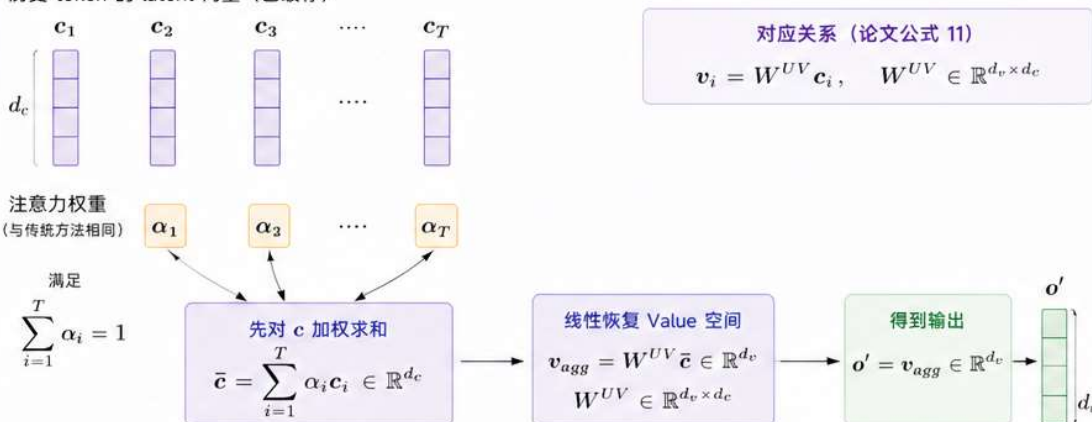
1. 传统注意力：用 Value 的加权和得到输出

历史 token (已缓存)



2. MLA: 用 latent 向量 c 先聚合, 再恢复输出

历史 token 的 latent 向量 (已缓存)



3. 数学推导：两种方式输出完全等价

由 $v_i = W^{UV} c_i$, 代入传统输出:

$$o = \sum_{i=1}^T \alpha_i v_i = \sum_{i=1}^T \alpha_i (W^{UV} c_i)$$

$$= W^{UV} \sum_{i=1}^T \alpha_i c_i = W^{UV} \bar{c}$$

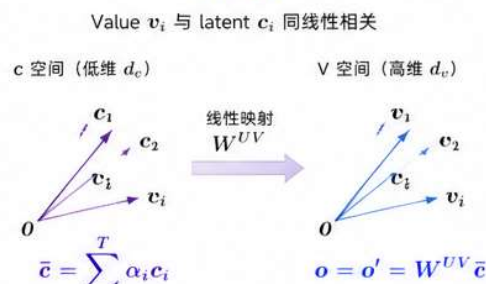
而 MLA 的输出: $o' = W^{UV} \bar{c}$

因此
 $o = o'$
两者完全等价!



4. 直观理解

几何视角

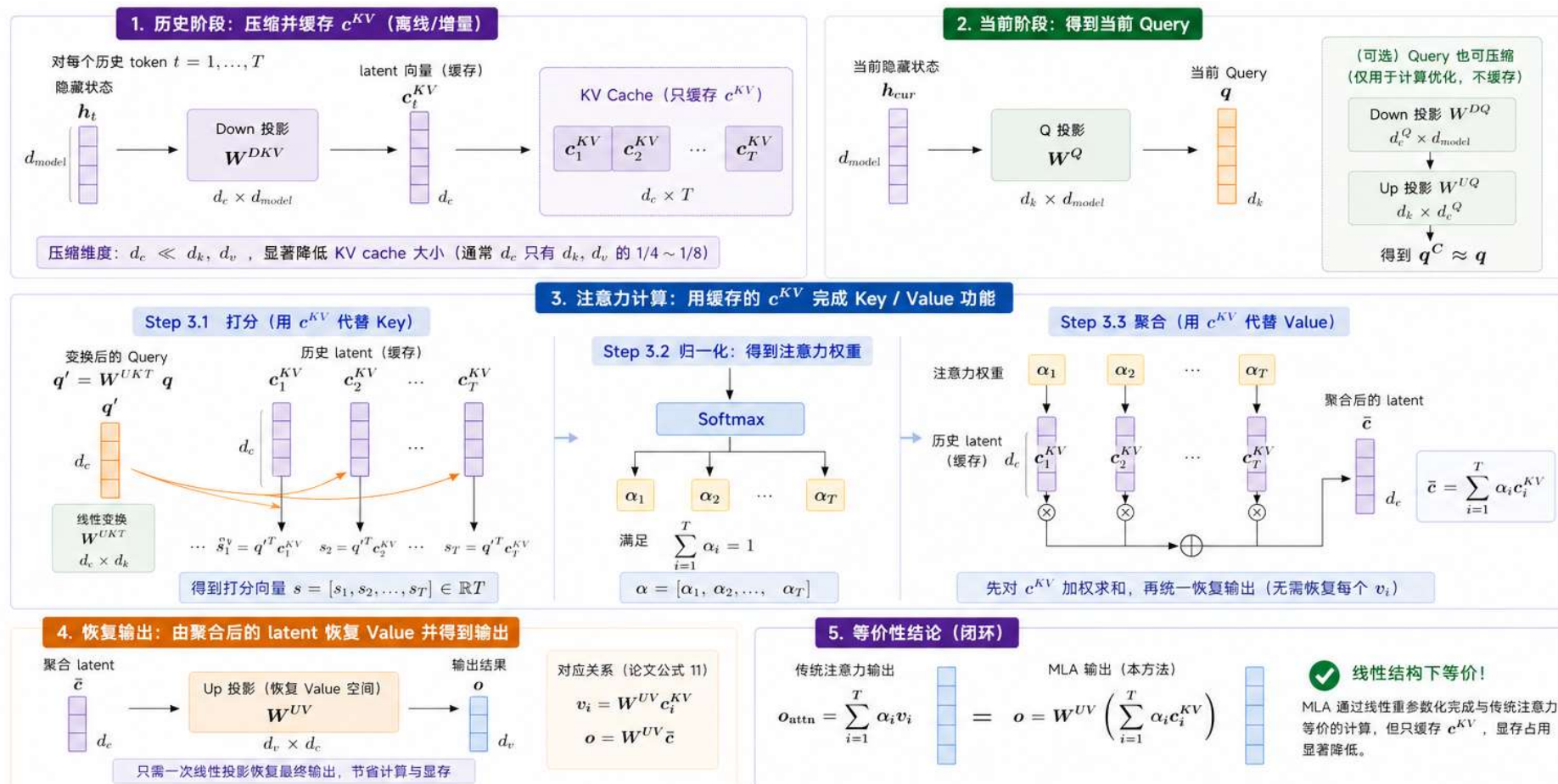


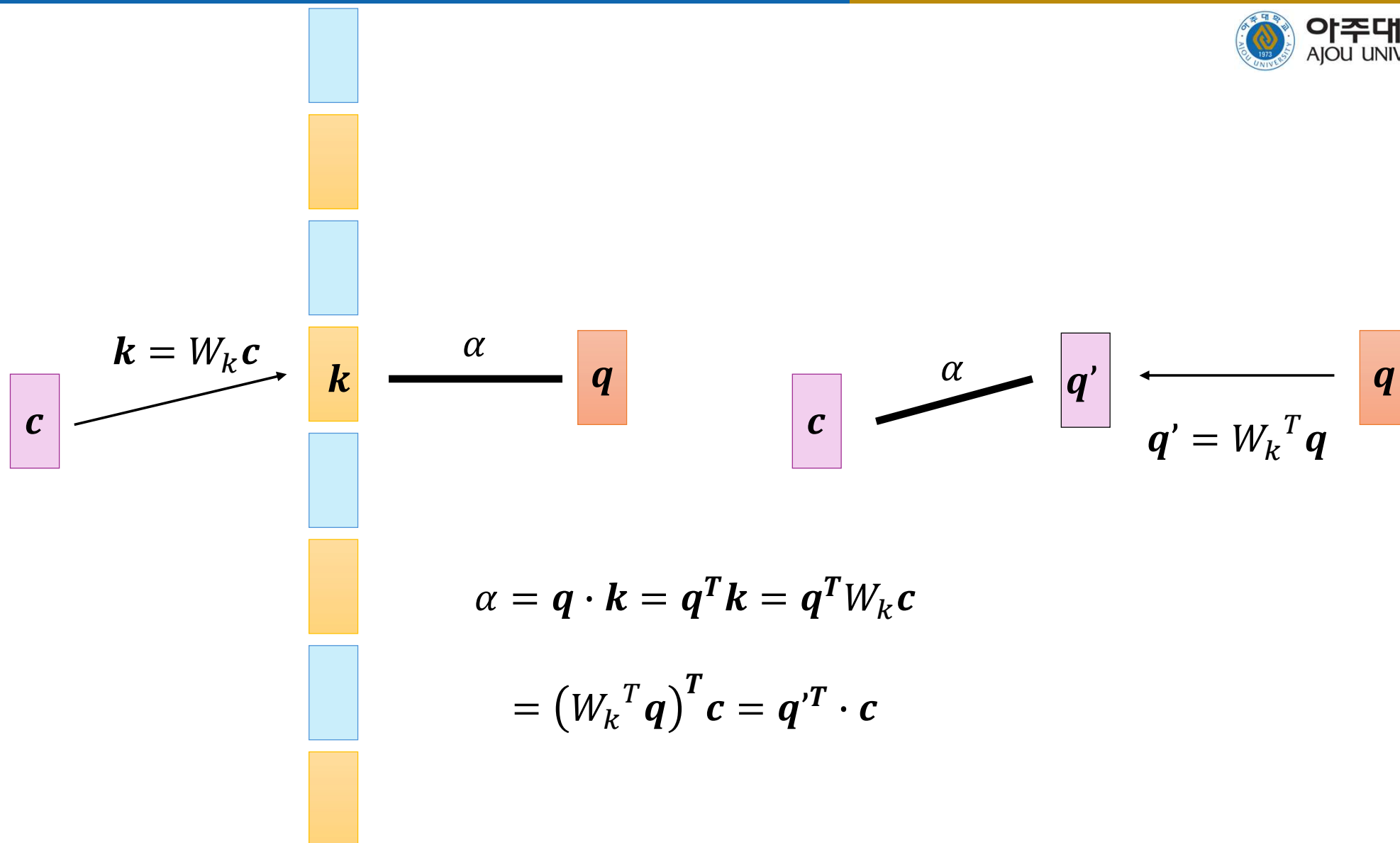
一句话理解

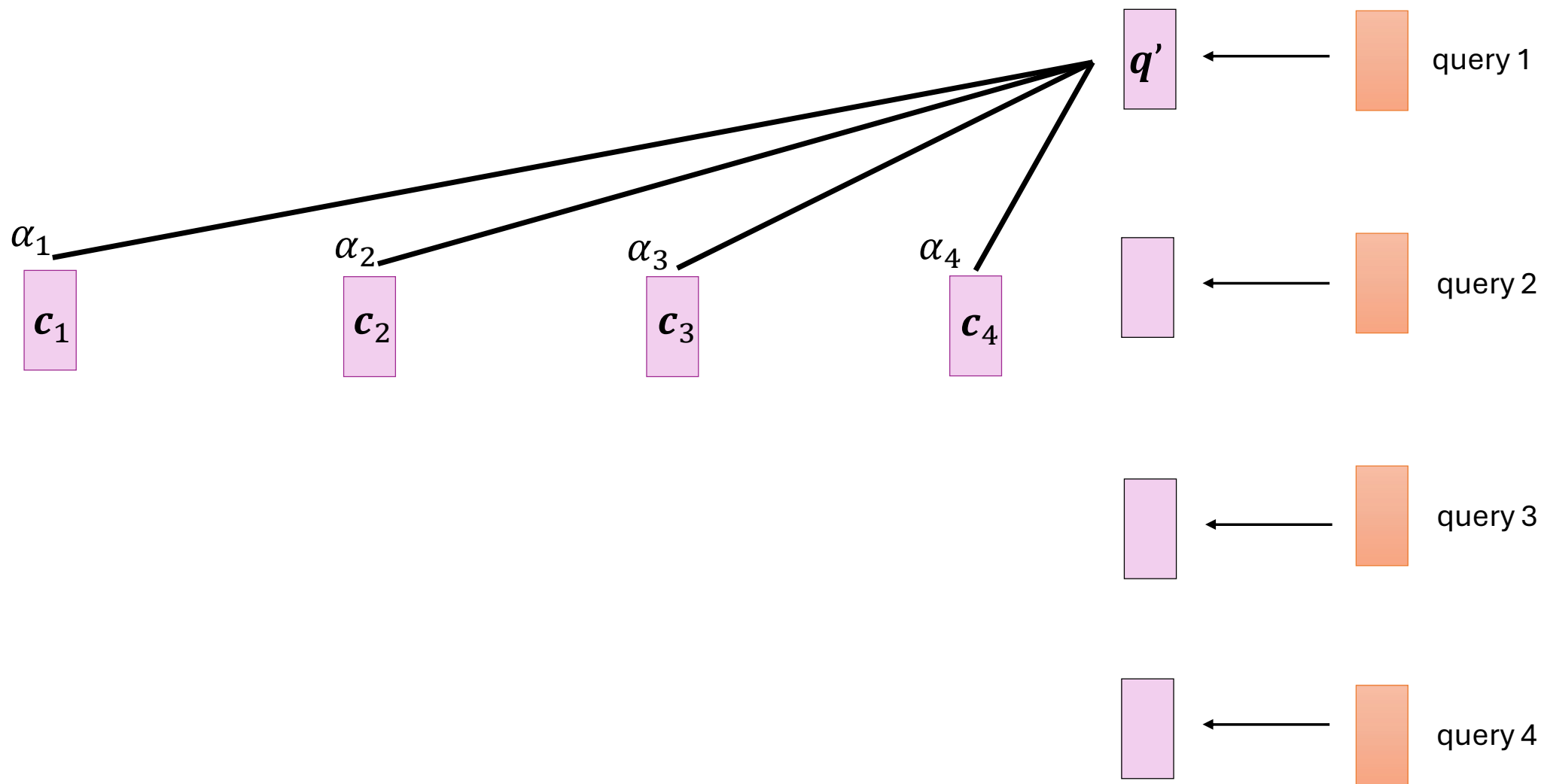
MLA 先在低维 c 空间完成“加权平均”,
再通过线性映射 W^{UV} 一次性
恢复到 Value 空间,
效果与直接对 Value 加权完全一致!

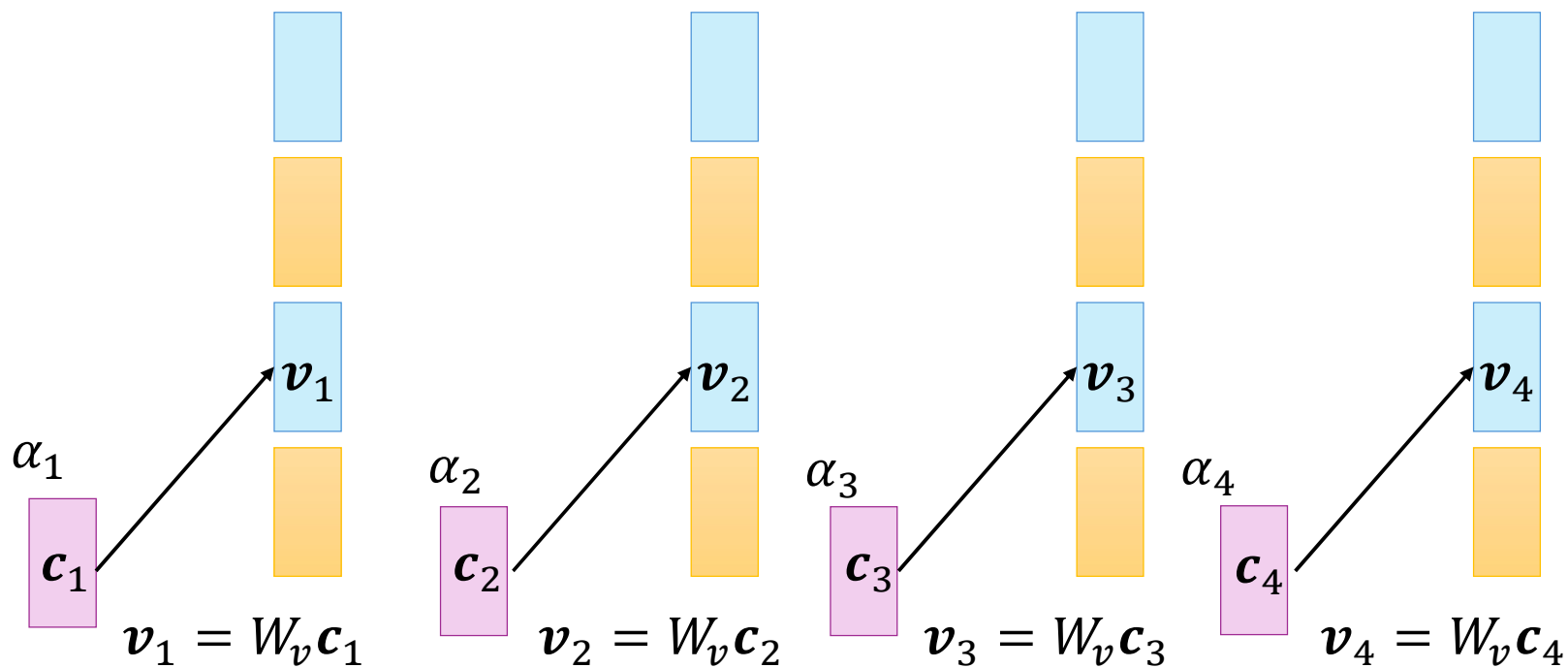


Multi-head Latent Attention





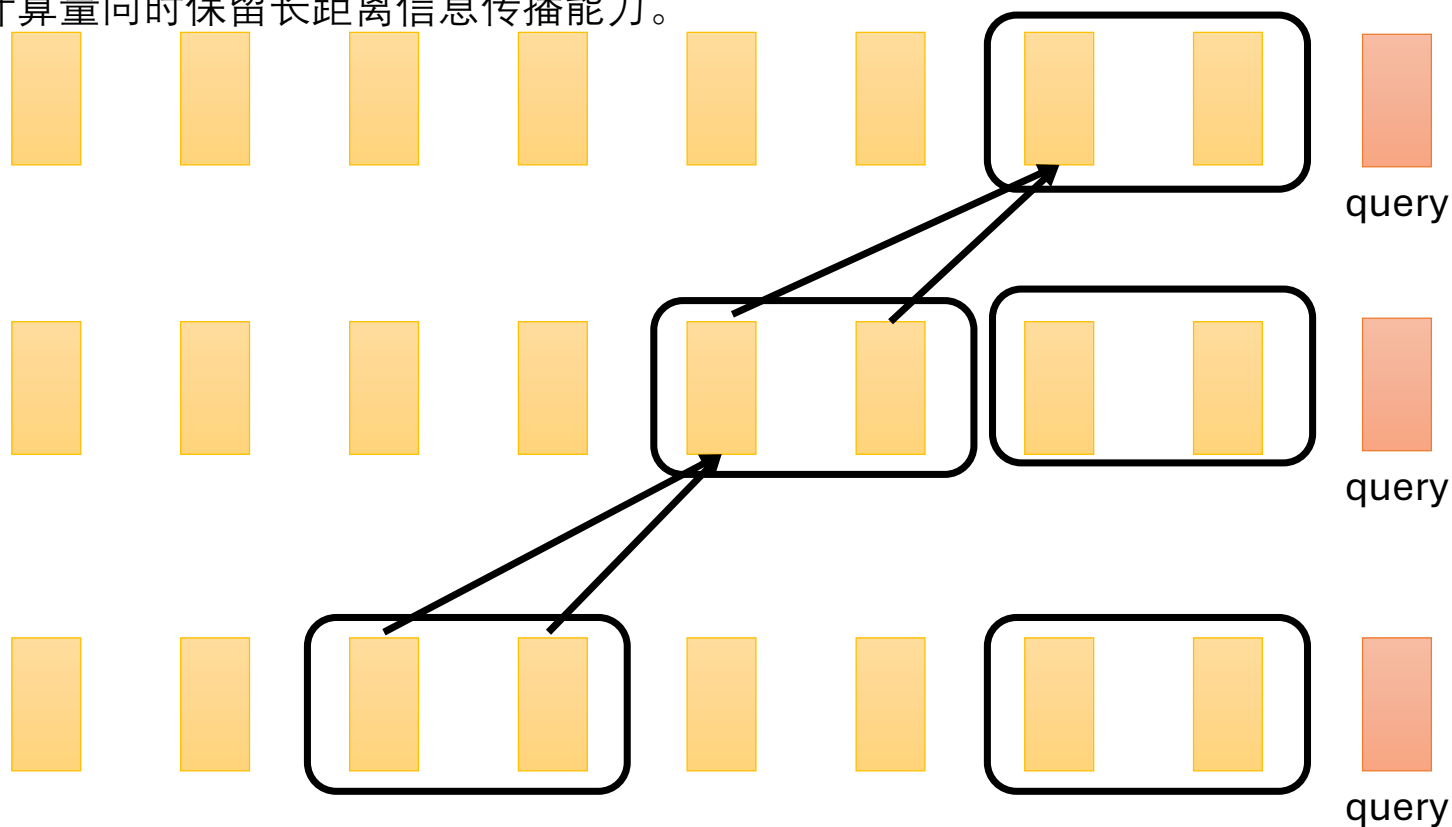




$$\begin{aligned}
 \mathbf{o} &= \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 + \alpha_4 \mathbf{v}_4 \\
 &= \alpha_1 W_v \mathbf{c}_1 + \alpha_2 W_v \mathbf{c}_2 + \alpha_3 W_v \mathbf{c}_3 + \alpha_4 W_v \mathbf{c}_4 \\
 &= W_v (\alpha_1 \mathbf{c}_1 + \alpha_2 \mathbf{c}_2 + \alpha_3 \mathbf{c}_3 + \alpha_4 \mathbf{c}_4)
 \end{aligned}$$

Sliding Window Attention

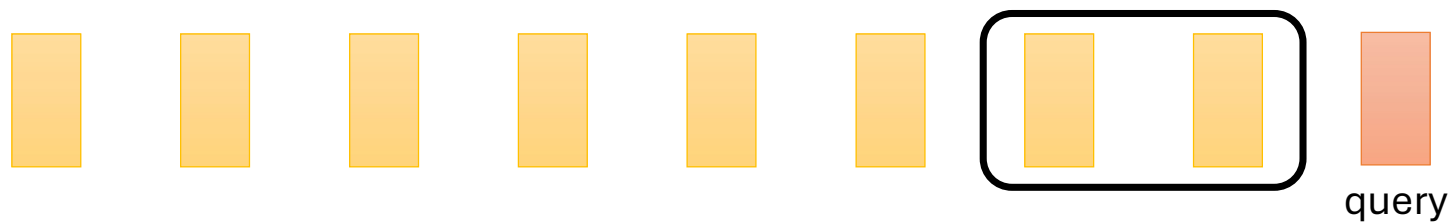
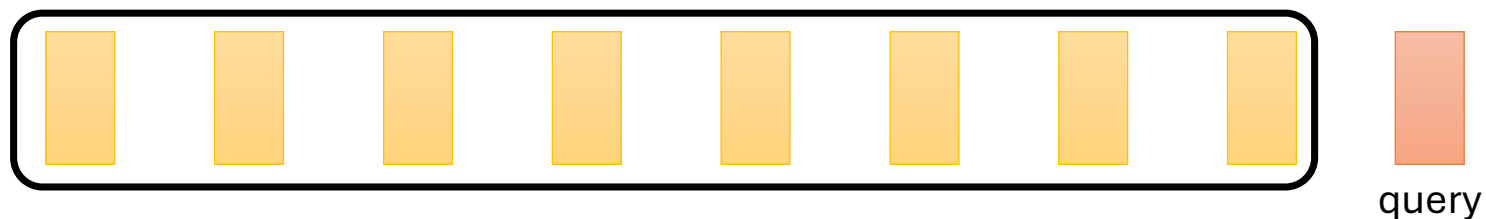
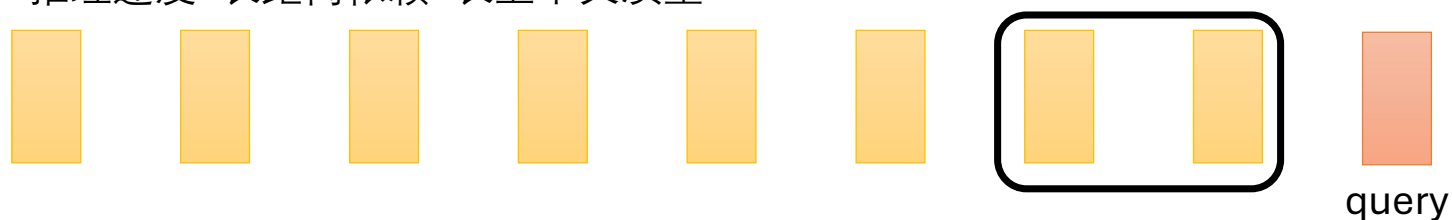
Mistral 7B 使用的 Sliding Window Attention 的机制，单层是局部注意力，多层堆叠后形成近似全局感受野，降低计算量同时保留长距离信息传播能力。



Mistral 7B

Sliding Window Attention

GPT-OSS (图中示意) 不是所有层都全局注意力, 大多数层: 局部窗口注意力; 少数层: 全局注意力
兼顾: 推理速度+长距离依赖+长上下文质量



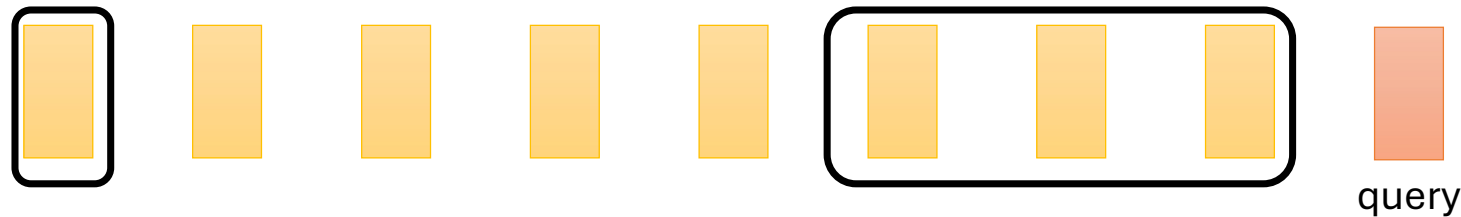
GPT-OSS

Streaming LLM (固定缓存的长上下文推理)

[固定前缀记忆]

+

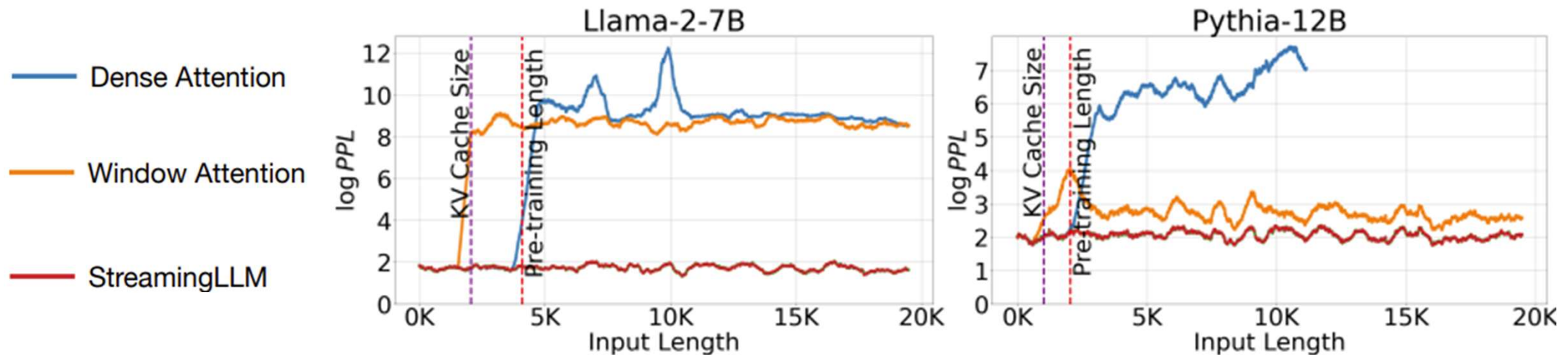
[最近上下文]



永久保留开头的 Token (Attention Sinks) : 通常只需保留前 4 个 Token。

保留最近的滑动窗口 Token : 例如最近的 2044 个。

丢弃中间的 Token : 那些既不是开头也不是最近的 Token。



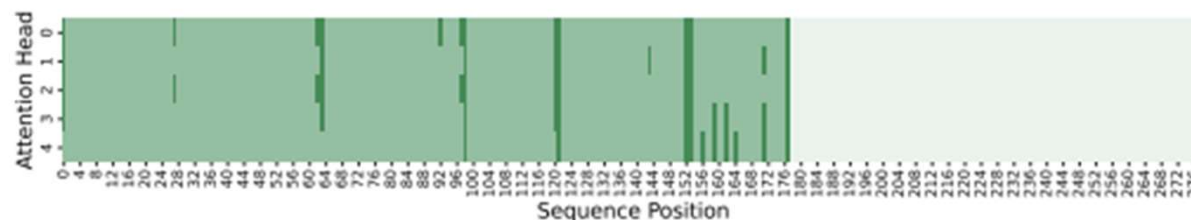
Pruning KV Cache

Scissorhands: <https://arxiv.org/abs/2305.17118>

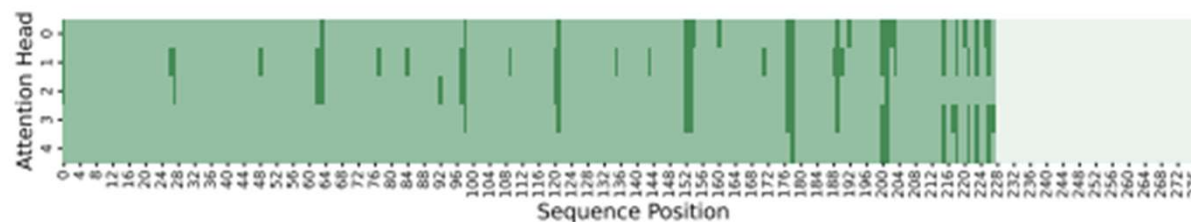
H2O: <https://arxiv.org/abs/2306.14048>

- 每次只有小部分的 Tokens 有 Attention
- 少数 token 会反复吸走大量 attention

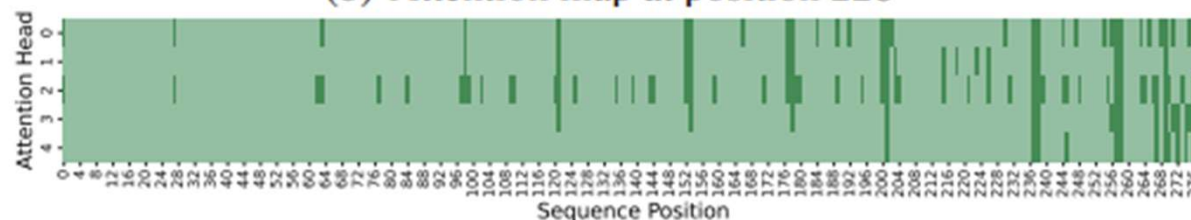
Scissorhands核心思想：找长期低 attention token，删掉 KV cache
H2O核心思想：保留高频热点 token (Heavy Hitters) + 最近 token



(a) Attention map at position 178



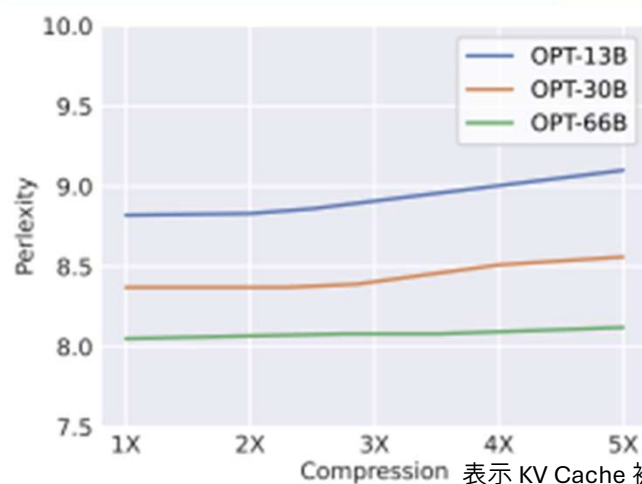
(b) Attention map at position 228



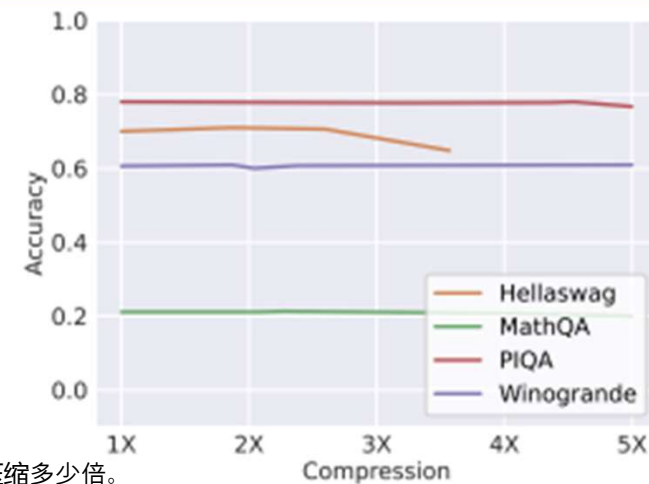
(c) Attention map at position 278

这张图展示 KV Cache 压缩后的效果变化。横轴是压缩倍率，纵轴是 perplexity 或准确率。可以看到，即使压缩到 3~5 倍，多数任务性能下降仍然有限。说明 KV Cache 存在冗余，优化空间很大。

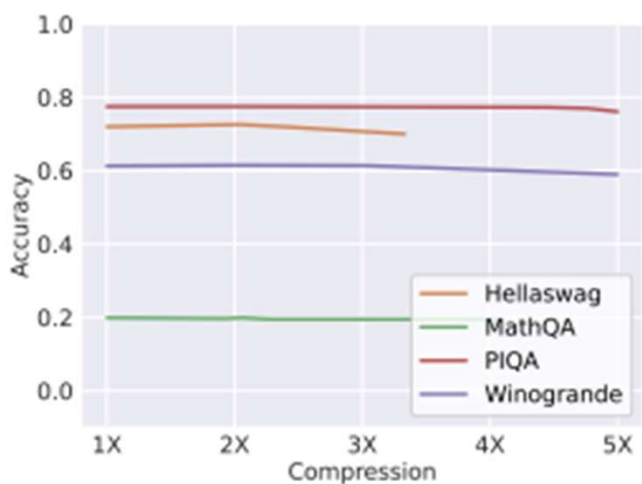
Scissorhands: <https://arxiv.org/abs/2305.17118>



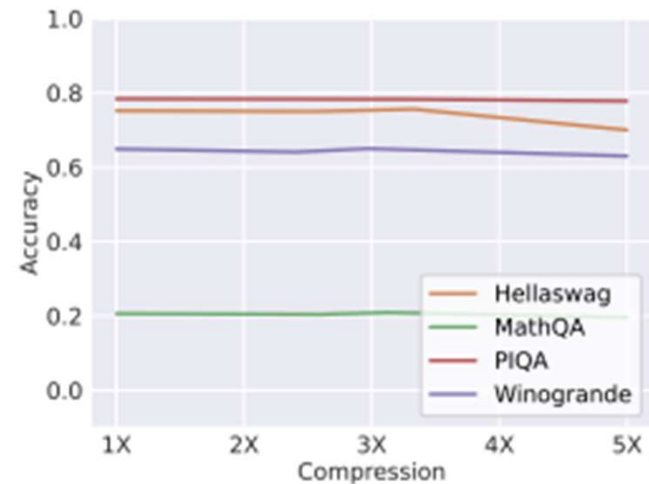
(a) Language Modeling



(b) OPT-6B Five shot



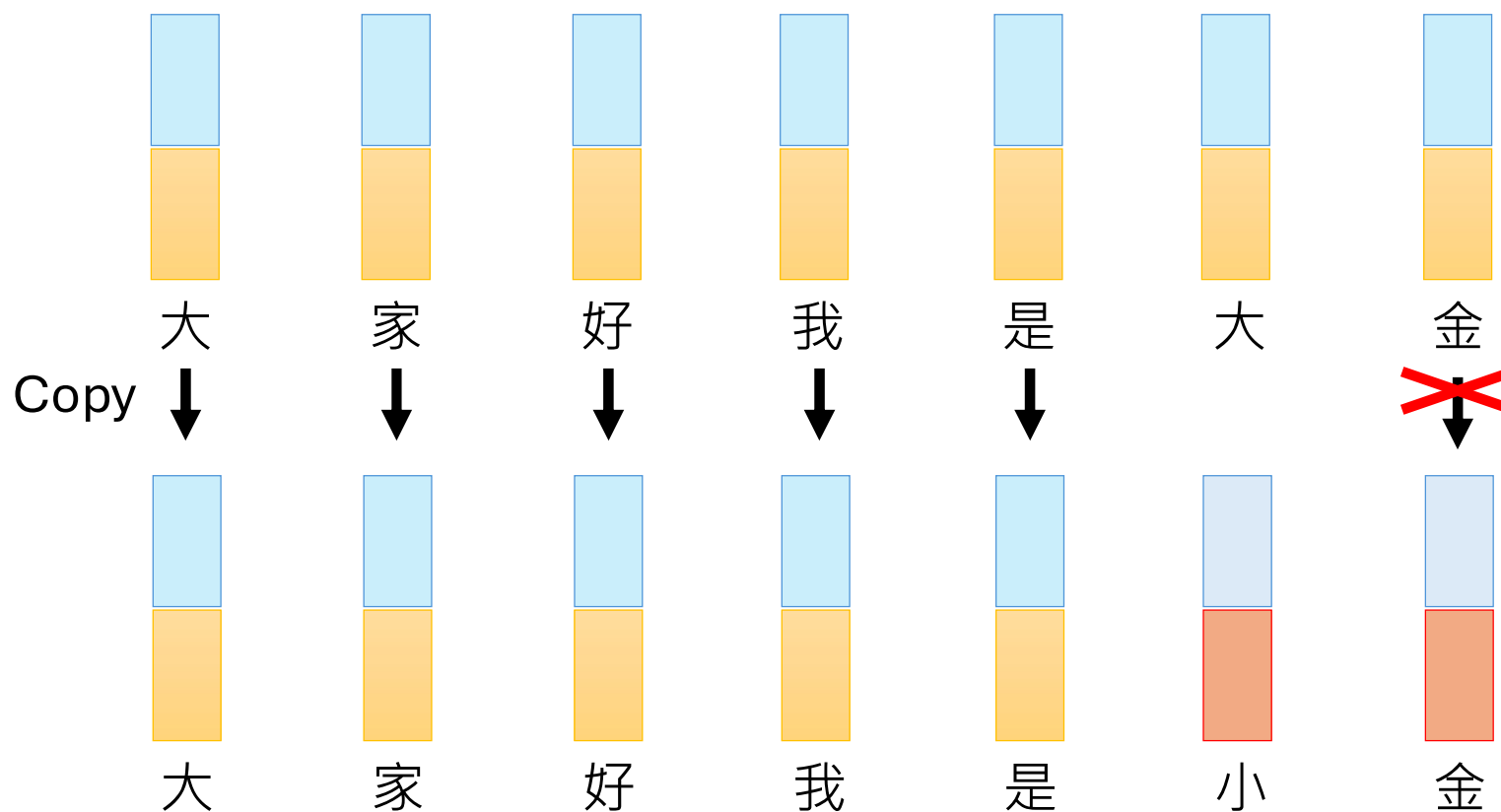
(c) OPT-13B Five shot



(d) OPT-30B Five shot

跨对话的 Cache

两次不同请求，如果前半段 prompt 相同，就不必重新计算前面 token 的 KV Cache，可以直接复用。



LLM推理的两阶段：Prefill与Decode

1 Prefill (预填充阶段)

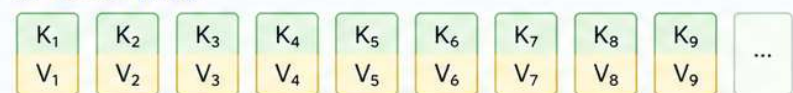
模型先把输入 prompt 的所有 token 一次性读完，并计算每个 token 的 Key 和 Value，存入 KV Cache。

输入 Prompt



Transformer
(多层)

KV Cache (存入)



为每个输入 token 计算并保存 Key (K) 和 Value (V)

特点

- 一次性处理所有输入 token，计算量大
- 主要耗时发生在这一阶段（尤其是长输入时）
- 结果是填满 KV Cache

2 Decode (解码生成阶段)

基于已有的 KV Cache，模型逐步生成输出 token：每次只生成 1 个新 token，并把它加入到 KV Cache 中。

已填充的
KV Cache



Transformer
(多层)

生成下一个
token

这 第 1 个输出 token

更新后的
KV Cache



Transformer
(多层)

生成下一个
token

篇 第 2 个输出 token

...

文章 第 t 个输出 token

特点

- 每次只生成 1 个 token，计算量小很多
- 利用已有 KV Cache，加速生成
- 将新生成的 token 的 K、V 加入缓存，继续下一步

Text tokens

Prices per 1M tokens.

Standard

Batch

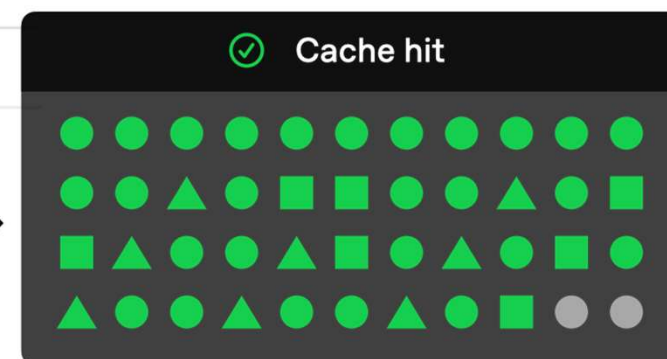
Flex

Priority

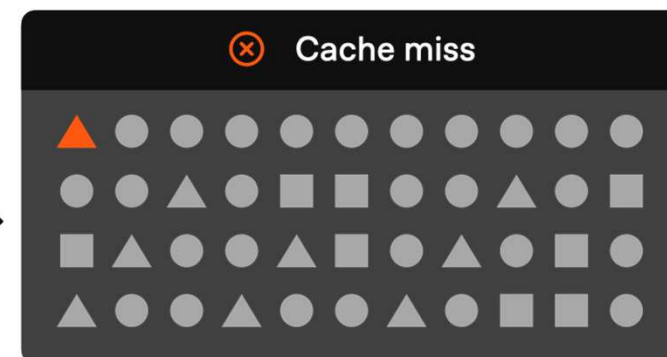
Our latest models

Model	Short context ⓘ			Long context ⓘ		
	Input	Cached input	Output	Input	Cached input	Output
gpt-5.4	\$2.50	\$0.25	\$15.00	\$5.00	\$0.50	\$22.50
gpt-5.4-pro	\$30.00	-	\$180.00	\$60.00	-	\$270.00

Cache Hit = 命中缓存，已有 KV Cache 可以直接复用。



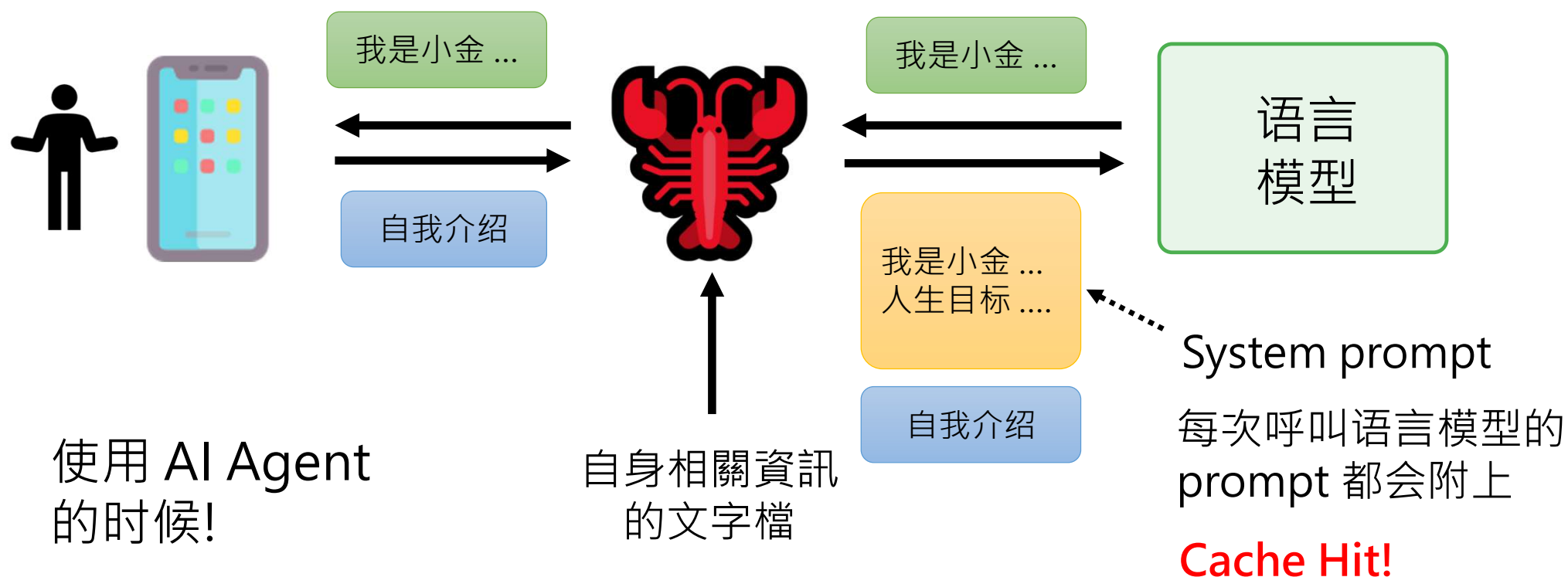
Cache Miss = 没命中缓存，需要重新计算。



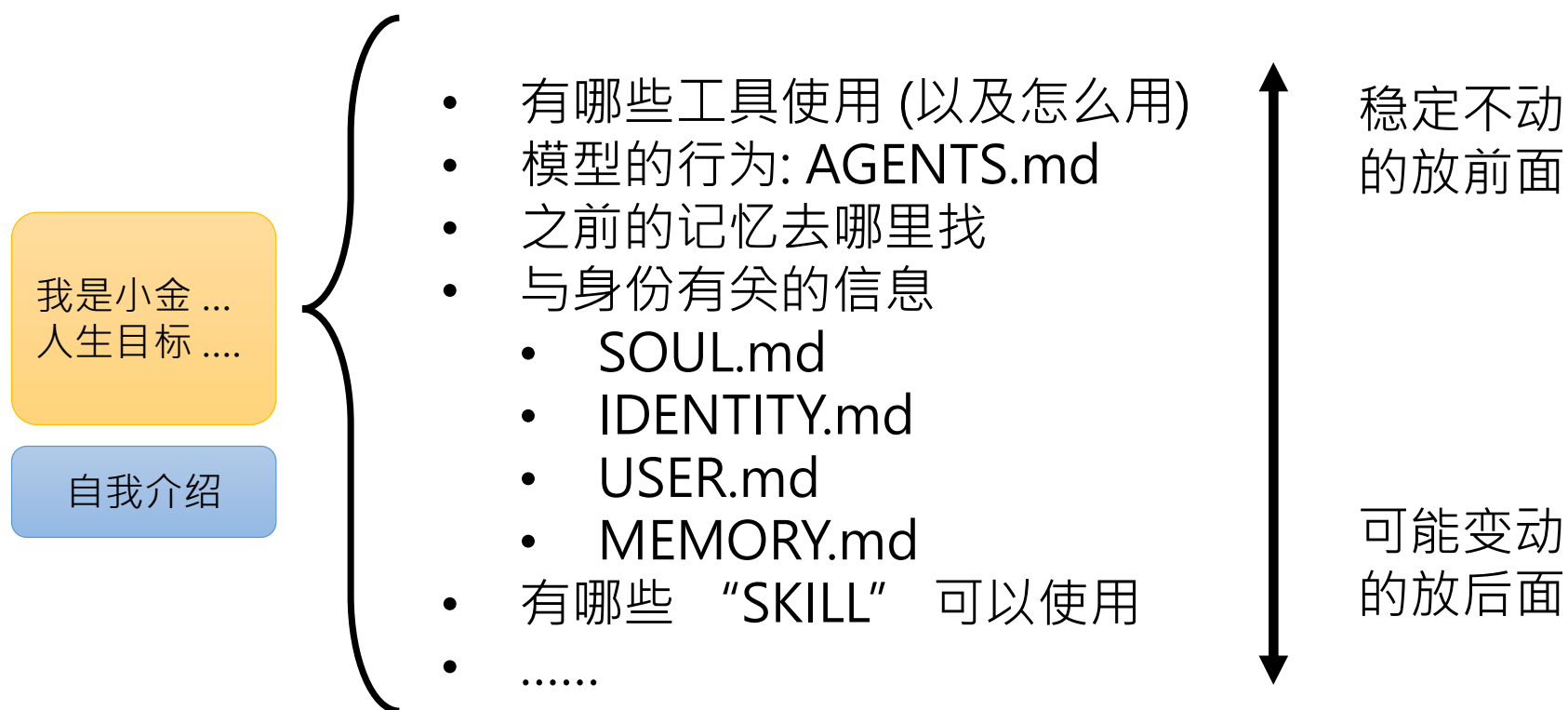
<https://developers.openai.com/api/docs/guides/prompt-caching/>
<https://developers.openai.com/api/docs/pricing>

什么时候跨对话的Cache可以发挥作用

当应用频繁向模型发送大量重复上下文（用户画像、系统提示词、长期记忆）时，可以复用之前的KV Cache，减少 Prefill 成本。AI Agent 是典型高命中场景。



什么时候跨对话的Cache可以发挥作用



Cache hit little



帮我订从仁川到上海的飞机

帮我订从旧金山到纽约的飞机

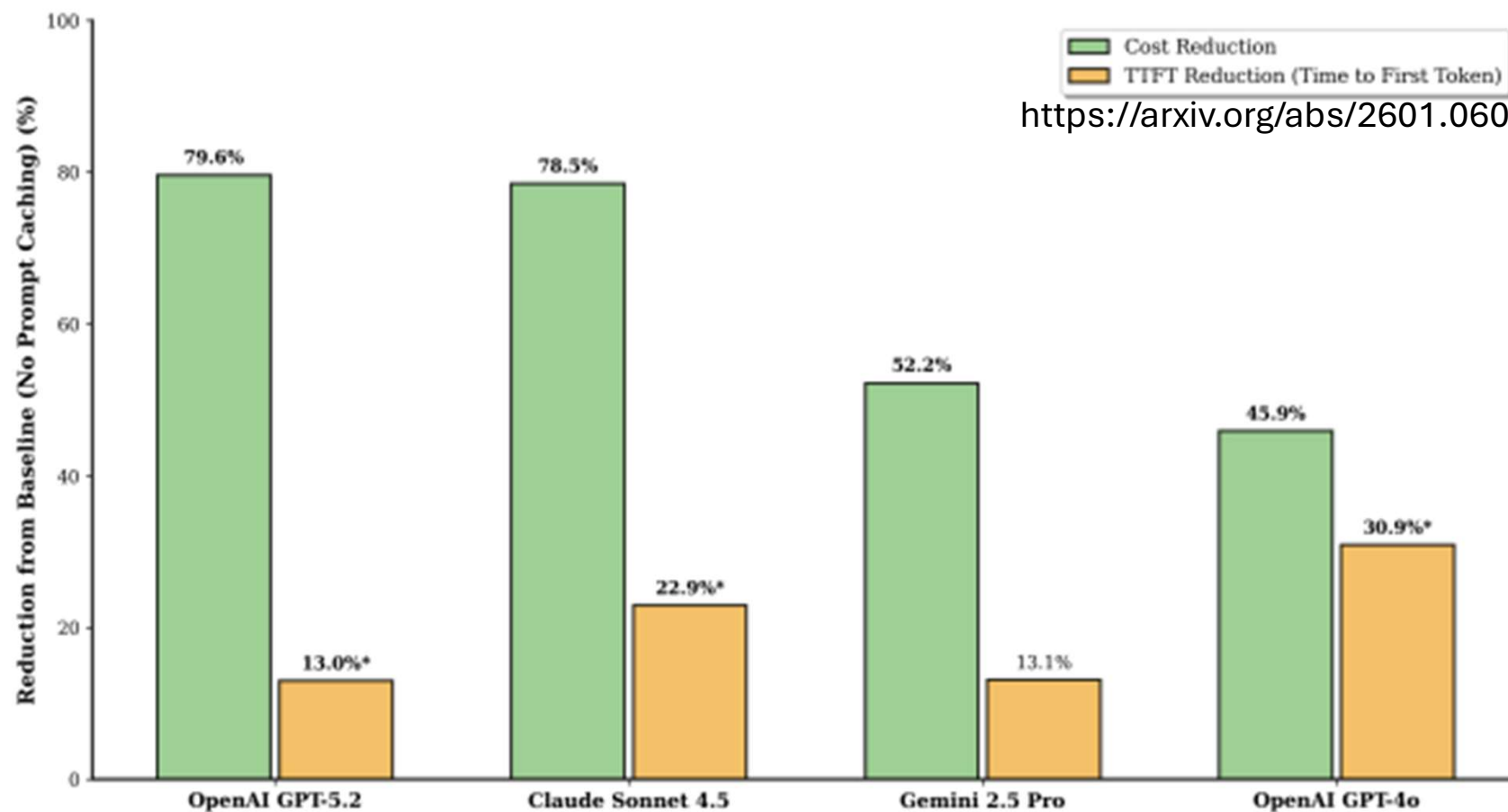
Cache hit more



帮我订从 x 到 y 的飞机 x=仁川, y=上海

帮我订从 x 到 y 的飞机 x=旧金山, y=纽约

Prompt Caching 带来的实际收益



<https://arxiv.org/abs/2601.06007>

结论

方法	方法说明	是否改变原有的 Attention	是否需要训练模型	其他代价
Flash Attention	少搬资料	X	X	一点额外运算 + 一点点烧脑
KV Cache	储存已经算出来的 key 和 value	X	X	占用记忆体
Multi-query Attention	多个 query 共享 key 和 value	O	O	可能明显伤害模型能力
Group-query Attention	多个 query 分组共享 key 和 value	O	O	较 MQA 好, 但仍有损失风险
Multi-head Latent Attention	压缩 key 和 value	O	O	需要额外训练与结构调整
Sliding Window Attention	改变 Attention 范围	O	?	长距离能力可能下降
Streaming LLM	改变 Attention 范围	O	?	需设计 sink token / 缓存策略
Pruning KV Cache	丢弃 key 和 value	O	X	可能明显伤害模型能力
Speculative Decoding	用小模型来预言生成结果	X (理论上)	X	小模型还是需要耗费额外算力



Mixture of Experts

专家混合模型

<https://arxiv.org/abs/2205.14135>

专家混合模型概述

专家混合 (MoE) 架构是如何工作的 ?

如何优化 MoE 内核 ?

如何对专家混合模型进行并行化 ?

回顾：Transformer 模块

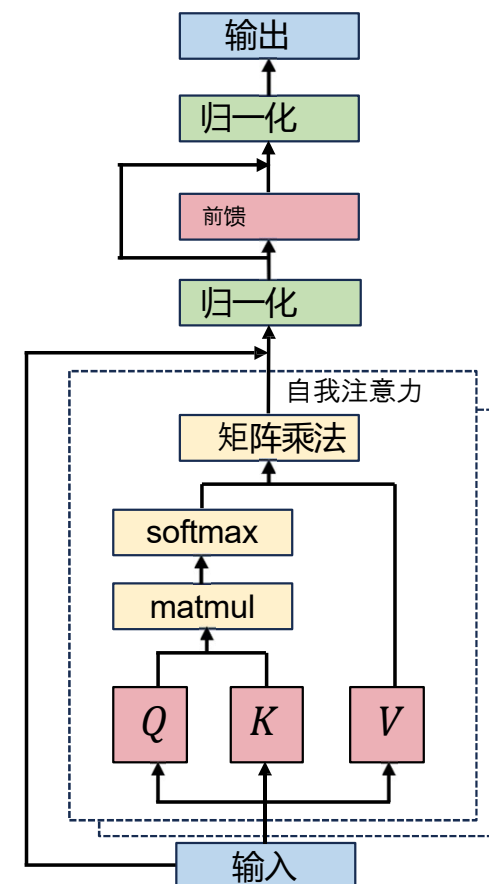
典型的Transformer模块

$$Z = \text{SelfAttention}(XW_K, XW_Q, XW_V)$$

$$Z = \text{LayerNorm}(X + Z)$$

$$H = \text{LayerNorm}(\text{ReLU}(ZW_1)W_2 + Z)$$

(多头) 自注意力机制，随后是一个线性层和ReLU层，
以及一些额外的残差连接和归一化操作



普通前馈层

前馈层

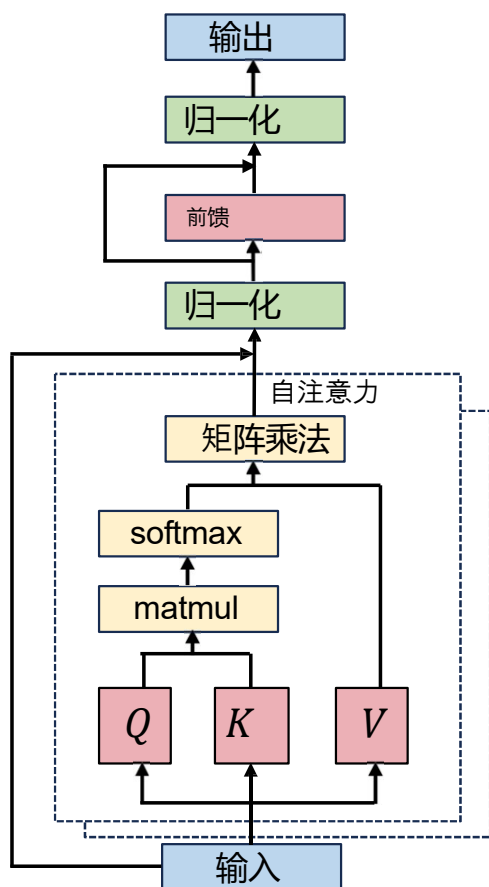
$$H = \text{LayerNorm}(\text{ReLU}(ZW_1)W_2) + Z$$

$$W_1 \in R^{n \times m}$$

特征维度的增加将导致计算量呈二次方增长

在前馈网络 (FFN) 层中 , 所有元素都被混合在一起

参数量最大的是前馈层 (**FFN**)

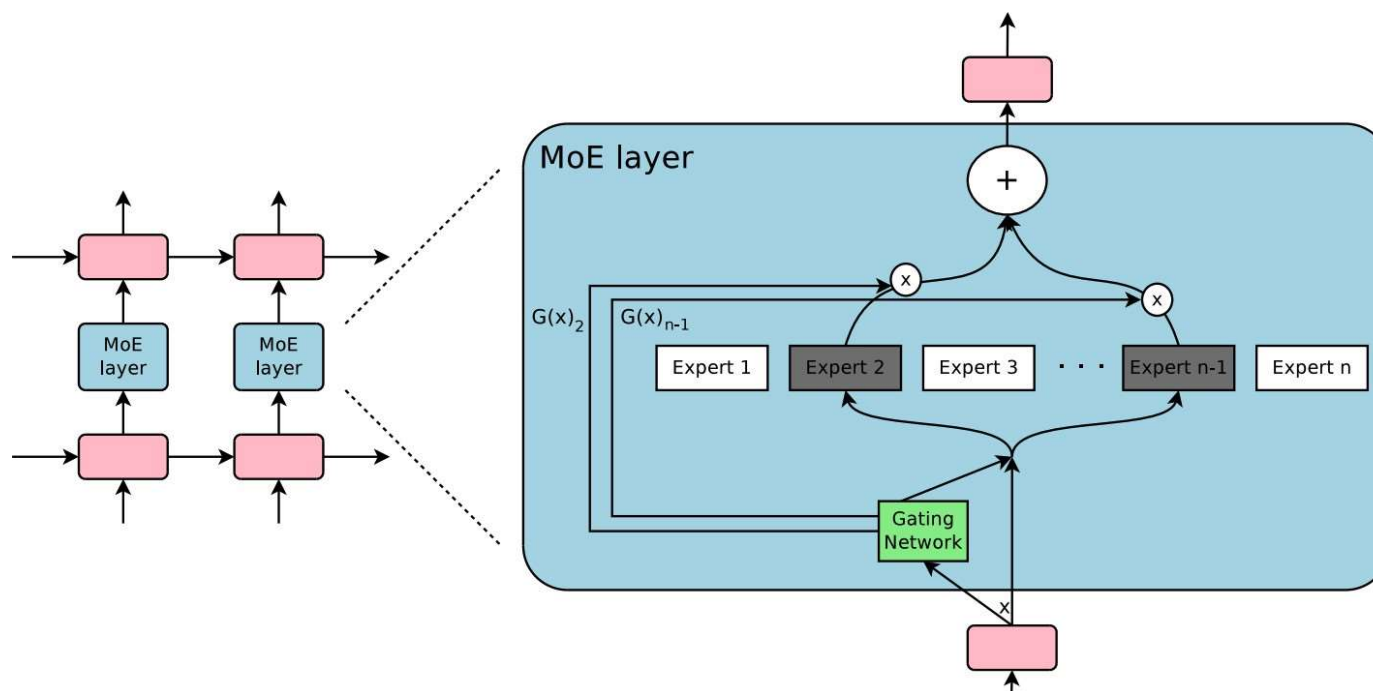


	GPT-3
vocab	50,257
d_model	12288
n_heads	96
n_layers	96
d_ff	49152
LayerNorm (B)	0.00
Embedding (B)	0.62
Attention (B)	57.99
Feed-forward (B)	115.97
Total (B)	174.57

专家混合模型

核心思想：引入多个专家，每个专家都是一个全连接输出层；让每个专家专注于预测特定子集案例的正确答案

优势：FLOPS 随模型规模呈亚线性增长，参数规模增长远快于计算量增长。



深入探讨专家混合模型

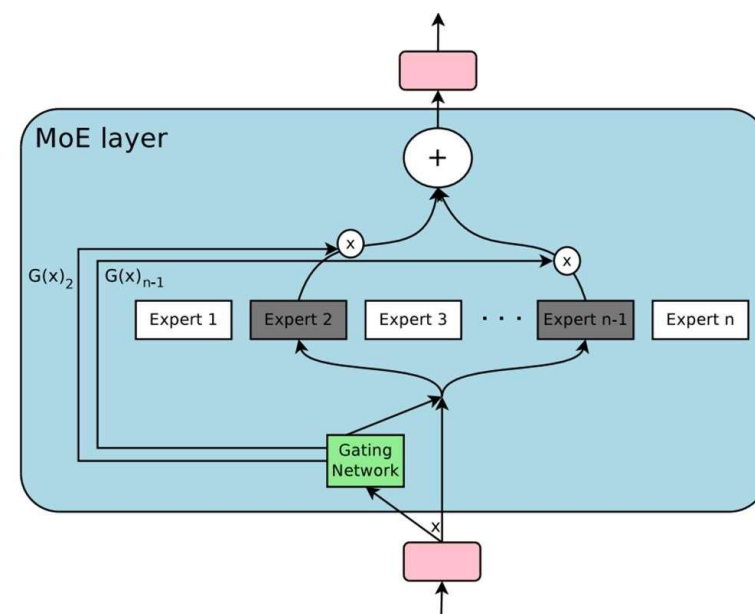
典型的MoE层（假设单实例并激活两个专家）

门控： $G = \text{Softmax}(W_G X)$
 专家权重： $I = \{i_0, i_1\} = \text{TopK}(G, k = 2)$
 输出权重： $s_0 = \frac{G_{i_0}}{(G_{i_0} + G_{i_1})}, s_1 = \frac{G_{i_1}}{(G_{i_0} + G_{i_1})}$
 输出： $Y = s_0 \text{FFN}_{i_0}(X) + s_1 \text{FFN}_{i_1}(X)$

从 N 个专家中贪婪地选取前 K 名

示例模型：Mixtral-8x7B 从八个专家中选取 2 个

不同的模型可能具有不同的FFN配置，通常包含多个线性层和一些非线性混合层



混合专家例题

设某个 token 的输入向量为：

$$X = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

MoE 层共有 **4 个专家**，采用：

TopK = 2

Router (门控网络) 权重矩阵为：

$$W_G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix}$$

每个专家都是两层前馈网络 (FFN)：

$$FFN_i(X) = W_{2,i} \text{ReLU}(W_{1,i}X)$$

Expert 1

$$W_{1,1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, W_{2,1} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Expert 2

$$W_{1,2} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, W_{2,2} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Expert 3

$$W_{1,3} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}, W_{2,3} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Expert 4

$$W_{1,4} = \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}, W_{2,4} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

混合专家例题

请依照真实 MoE 流程求最终输出 Y

步骤要求：

1. 计算 Router logits：

$$G = \text{Softmax}(W_G X)$$

2. 取 Top-2 专家：

$$I = \text{TopK}(G, k = 2)$$

3. 重新归一化权重：

$$s_0 = \frac{g_{i_0}}{g_{i_0} + g_{i_1}}, s_1 = \frac{g_{i_1}}{g_{i_0} + g_{i_1}}$$

4. 分别计算被选中专家的 FFN 输出：

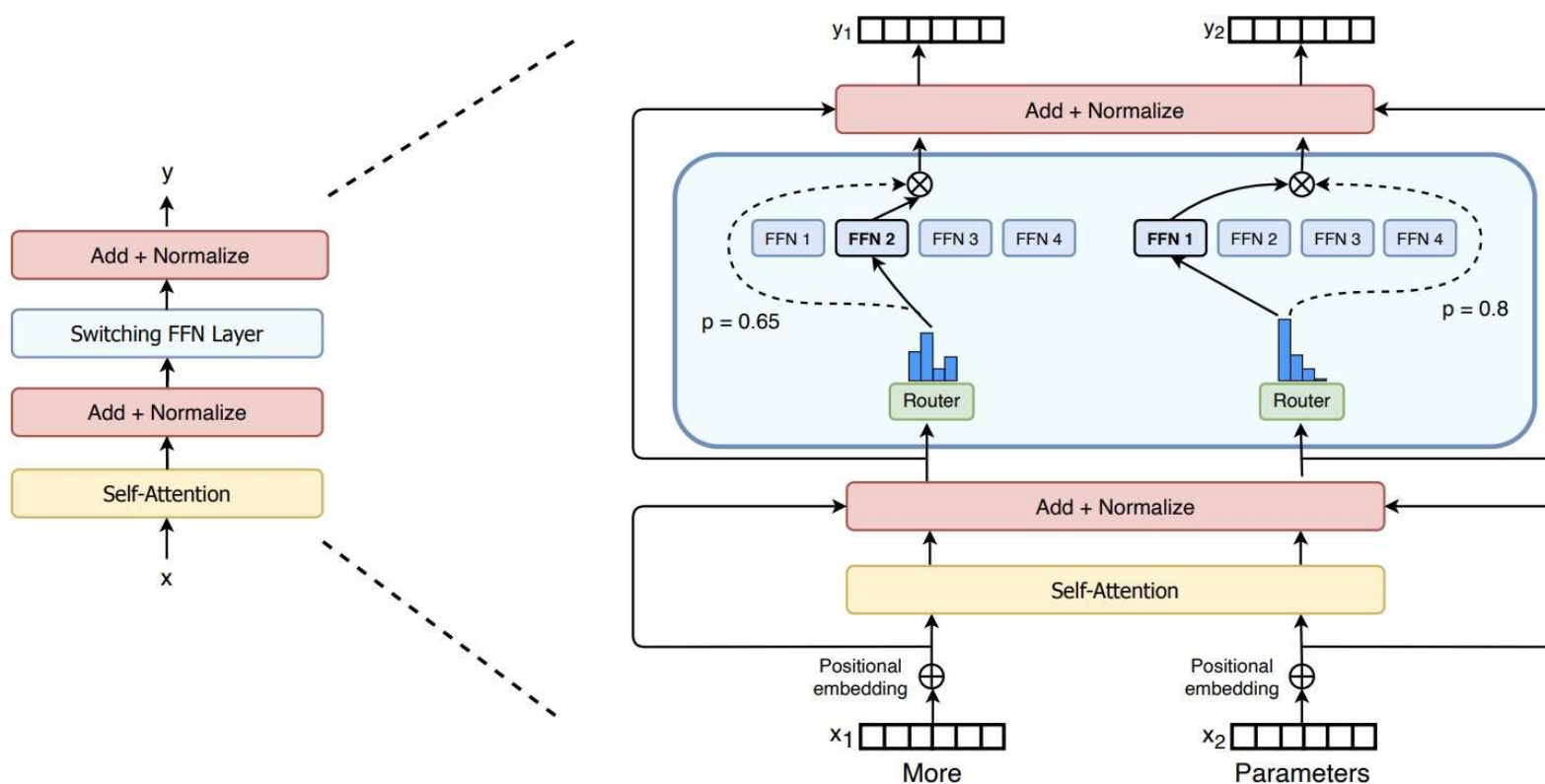
$$\text{FFN}_i(X) = W_{2,i} \text{ReLU}(W_{1,i} X)$$

5. 求最终输出：

$$Y = s_0 \text{FFN}_{i_0}(X) + s_1 \text{FFN}_{i_1}(X)$$

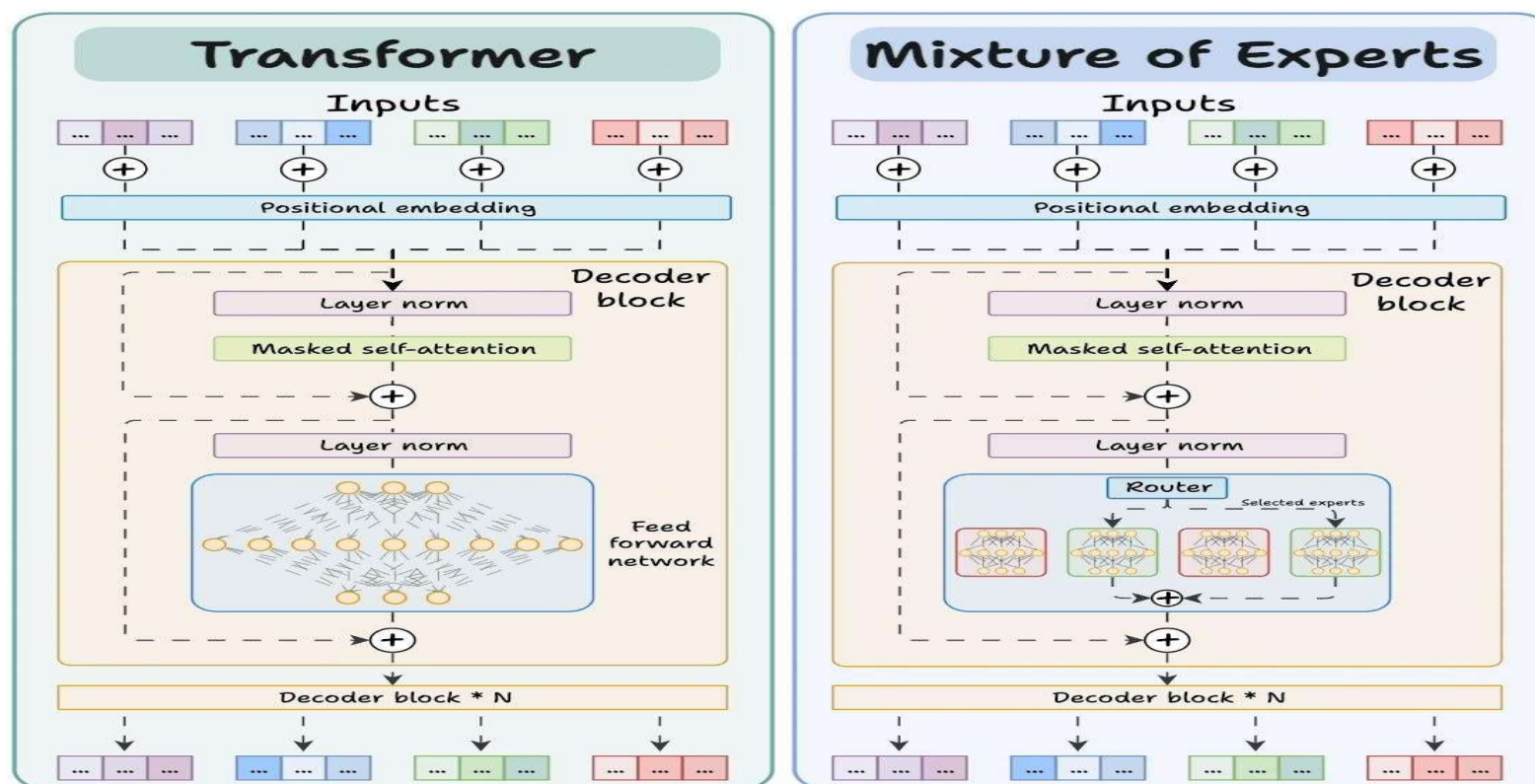
Transformer模型 + 专家混合模型

只需将Transformer模型中的全连接层替换为专家混合模型



Transformer + 专家混合模型

- 用专家混合模型替换全连接层



MoE在Transformer模型中的潜力

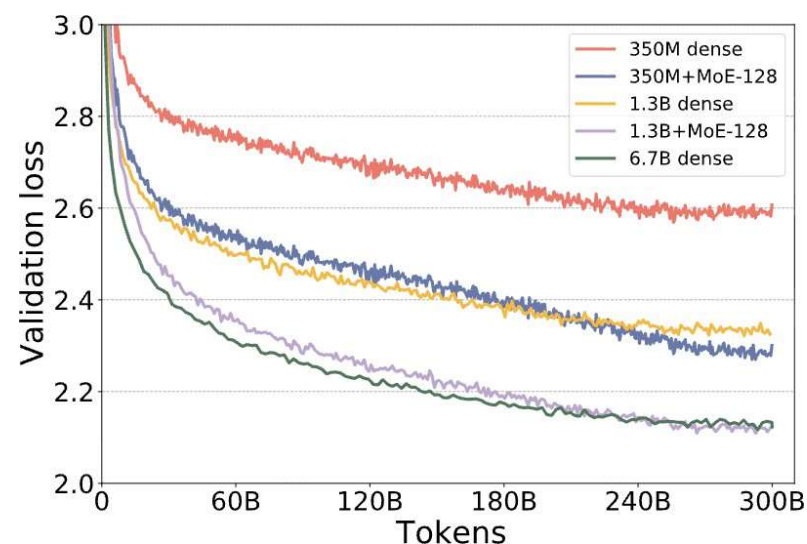
GLaM

- 1.3万亿参数的MoE
- 参数数量是GPT-3的7倍
- 计算量仅为 GPT-3 的二分之一

		GPT-3	GLaM	relative
cost	FLOPs / token (G)	350	180	-48.6%
	Train energy (MWh)	1287	456	-64.6%
accuracy on average	Zero-shot	56.9	62.7	+10.2%
	One-shot	61.6	65.5	+6.3%
	Few-shot	65.2	68.1	+4.4%

DeepSpeed MoE模型

- 67亿参数的密集型大语言模型 = 520亿参数的MoE模型 (含13亿个激活层)
- 训练计算量减少5倍



GLaM：基于专家混合模型的语言模型高效扩展

DeepSpeed-MoE：推进专家混合推理与训练，助力下一代人工智能规模化发展

案例研究：DeepSeek-V3 中的专家混合模型

- **共享专家**：每个token均通过（无路由）
- **路由专家**：由路由器（门控网络）针对每个token有选择地激活

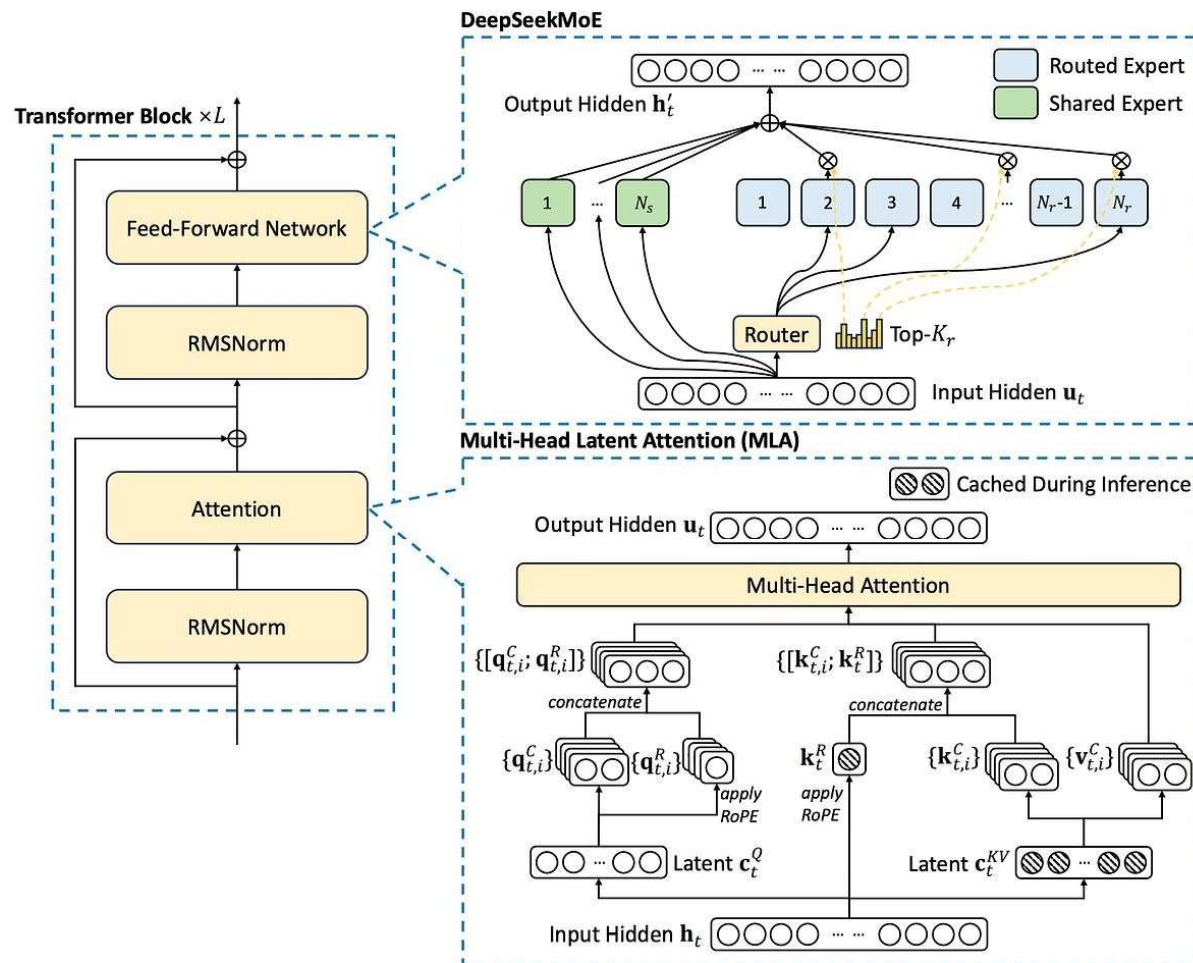
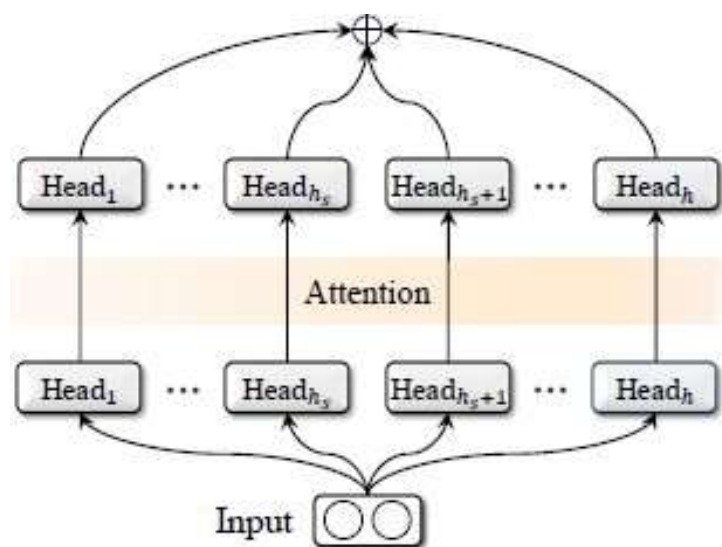


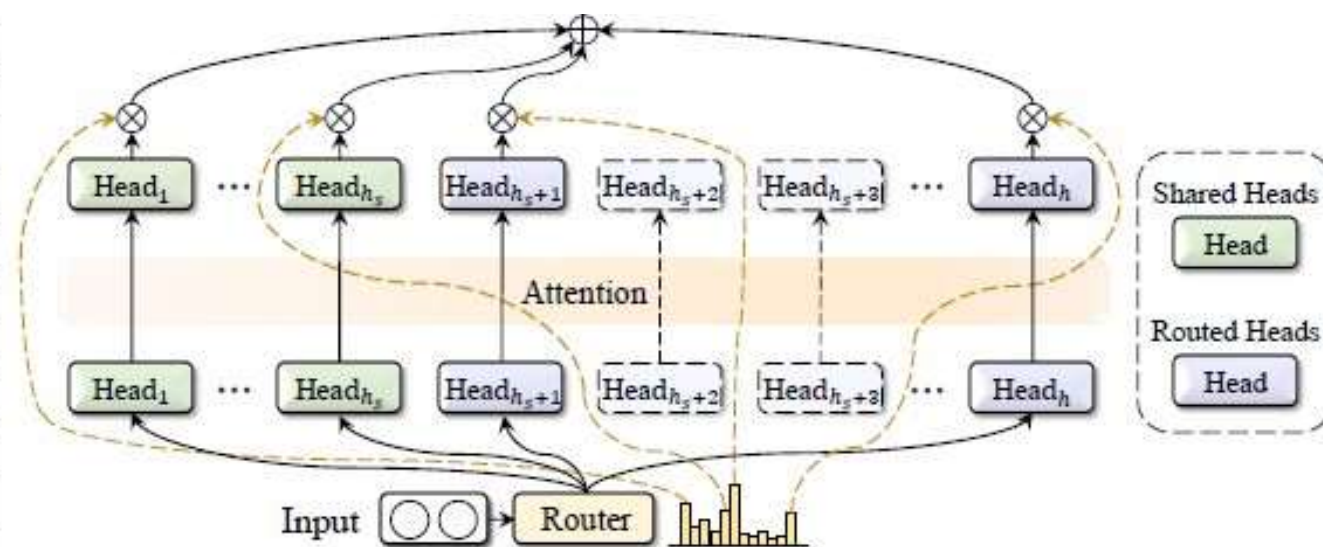
Figure 2 | Illustration of the basic architecture of DeepSeek-V3. Following DeepSeek-V2, we adopt MLA and DeepSeekMoE for efficient inference and economical training.

案例研究：混合头部注意力

- **共享头部**：每个token都会经过
- **路由头**：由路由器选择性激活



(a) Multi-Head Attention



(b) Our proposed Mixture-of-Head Attention

输入 token :

$$X = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

模型包含 :

1 个 共享头部

3 个 路由头部

TopK = 2

激活函数 = ReLU

Router

$$W_G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

Shared Expert (FFN)

$$Shared(X) = W_2^{(s)} ReLU(W_1^{(s)} X)$$

$$W_1^{(s)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, W_2^{(s)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Routed Expert 1

$$W_1^{(1)} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, W_2^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Routed Expert 2

$$W_1^{(2)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, W_2^{(2)} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Routed Expert 3

$$W_1^{(3)} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, W_2^{(3)} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

求最终输出

$$Y = Shared(X) + s_0 E_{i_0}(X) + s_1 E_{i_1}(X)$$